

Parallel Algorithms



Molecular Dynamics Simulations / N-Body

João L. Sobral

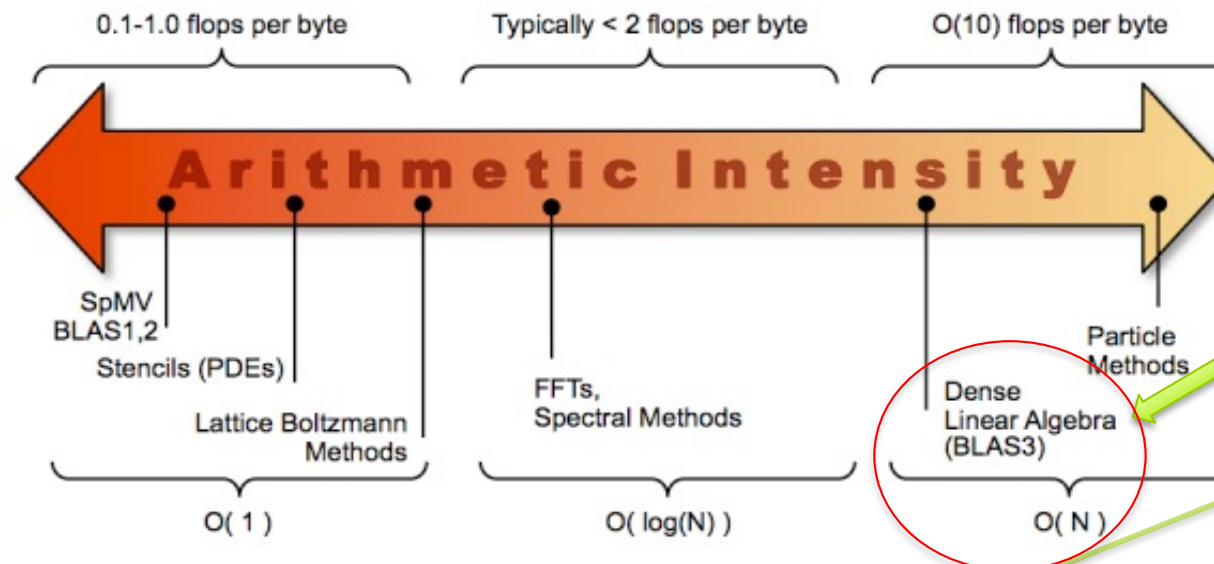
jls@di ...

Parallel Algorithms – Matrix multiplication

(recap-1) Why MatMult can get high performance

data moves

- **Arithmetic Intensity (operations per ~~byte loaded~~ byte)**



- **Data storage**

- **Regular** : vector, **matrix**
- Irregular (pointer based): tree, graphs

Data accesses
- **Regular** / irregular

Parallel Algorithms – Matrix multiplication

(recap-2) How to get high performance

□ Taking advantage of **temporal data locality**

- Generic Rationale (AXPY example)

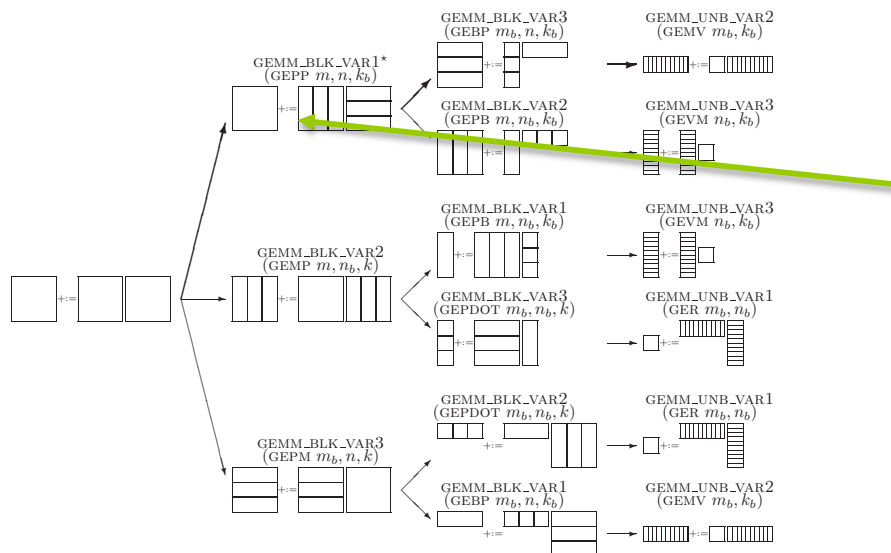
$$\begin{matrix} Mc \times N & Mc \times Kc & Kc \times N \\ \boxed{C} & := & \boxed{A} \boxed{B} \end{matrix}$$

Assumming $c = Mc \approx Kc$

$$\frac{2c^2n}{c^2 + 3cn} \frac{\text{flops}}{\text{memops}} \approx \frac{2c}{3} \frac{\text{flops}}{\text{memops}} \text{ when } c \ll n$$

e.g., $c=1 \Rightarrow 2/3$; $c=4 \Rightarrow 2.7$; **$c=128 \Rightarrow 85!$**

- **High-level view:** matrix multiplication can be decomposed into smaller matrix multiplications to address multiple levels of locality

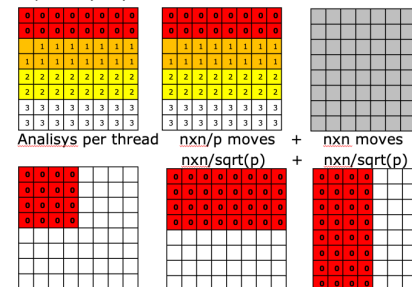


Parallelism (& block decomposition)

1. Data decomposition into independent blocks (due to **regular** data accesses)
2. But: DOT is more difficult to parallelize
3. Square blocks are better (less **data moves**)

■ Matrix partition

- By rows (cols) VS blocks



$$\begin{matrix} N^2 \\ \text{VS} \\ N^2/\text{sqrt}(P) \end{matrix}$$

Decreases 3
with P

Parallel Algorithms – Matrix multiplication

(recap-3) Leveraging matrix multiplication

Building block for other algorithms

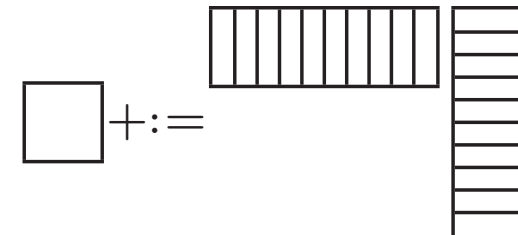
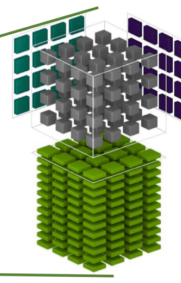
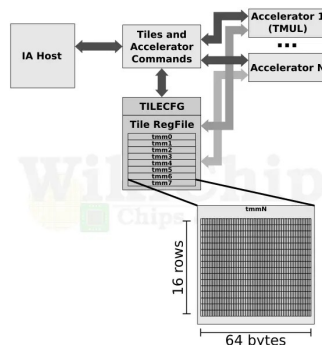
Algorithm: $A := \text{LU_UNB}(A)$ Partition $A \rightarrow \begin{pmatrix} A_{TL} & A_{TR} \\ A_{BL} & A_{BR} \end{pmatrix}$ where A_{TL} is 0×0 while $n(A_{TL}) < n(A)$ do Repartition $\begin{pmatrix} A_{TL} & A_{TR} \\ A_{BL} & A_{BR} \end{pmatrix} \rightarrow \begin{pmatrix} A_{00} & a_{01} & A_{02} \\ a_{10}^T & \alpha_{11} & a_{12}^T \\ A_{20} & a_{21} & A_{22} \end{pmatrix}$ where α_{11} is a scalar Variant 1: $a_{01} := L_{00}^{-1} a_{01}$ (TRSV) $a_{10}^T := a_{10}^T U_{00}^{-1}$ (TRSV) $\alpha_{11} := \alpha_{11} - a_{10}^T a_{01}$ (APDOT) Variant 2: $a_{10}^T := a_{10}^T U_{00}^{-1}$ (TRSV) $\alpha_{11} := \alpha_{11} - a_{10}^T a_{01}$ (APDOT) $a_{12}^T := a_{12}^T - a_{10}^T A_{02}$ (GEMV) Variant 3: $a_{01} := L_{00}^{-1} a_{01}$ (TRSV) $\alpha_{11} := \alpha_{11} - a_{10}^T a_{01}$ (APDOT) $a_{21} := (a_{21} - A_{20} a_{01}) / \alpha_{11}$ (GEMV, INVSCL) Variant 4: $\alpha_{11} := \alpha_{11} - a_{10}^T a_{01}$ (APDOT) $a_{21} := (a_{21} - A_{20} a_{01}) / \alpha_{11}$ (GEMV, INVSCL) $a_{12}^T := a_{12}^T - a_{10}^T A_{02}$ (GEMV) Variant 5: $a_{21} := a_{21} / \alpha_{11}$ (INVSCL) $A_{22} := A_{22} - a_{21} a_{12}^T$ (GER) Continue with $\begin{pmatrix} A_{TL} & A_{TR} \\ A_{BL} & A_{BR} \end{pmatrix} \leftarrow \begin{pmatrix} A_{00} & a_{01} & A_{02} \\ a_{10}^T & \alpha_{11} & a_{12}^T \\ A_{20} & a_{21} & A_{22} \end{pmatrix}$ endwhile	Algorithm: $A := \text{LU_BLK}(A)$ Partition $A \rightarrow \begin{pmatrix} A_{TL} & A_{TR} \\ A_{BL} & A_{BR} \end{pmatrix}$ where A_{TL} is 0×0 while $n(A_{TL}) < n(A)$ do Determine block size n_b Repartition $\begin{pmatrix} A_{TL} & A_{TR} \\ A_{BL} & A_{BR} \end{pmatrix} \rightarrow \begin{pmatrix} A_{00} & A_{01} & A_{02} \\ A_{10} & A_{11} & A_{12} \\ A_{20} & A_{21} & A_{22} \end{pmatrix}$ where A_{11} is $n_b \times n_b$ Variant 1: $A_{01} := L_{00}^{-1} A_{01}$ (TRSM) $A_{10} := A_{10} U_{00}^{-1}$ (TRSM) $A_{11} := \text{LU_UNB}(A_{11} - A_{10} A_{01})$ (GEMM, LU) Variant 2: $A_{10} := A_{10} U_{00}^{-1}$ (TRSM) $A_{11} := \text{LU_UNB}(A_{11} - A_{10} A_{01})$ (GEMM, LU) $A_{12} := A_{12} - A_{10} A_{02}$ (GEMM) Variant 3: $A_{01} := L_{00}^{-1} A_{01}$ (TRSM) $A_{11} := \text{LU_UNB}(A_{11} - A_{10} A_{01})$ (GEMM, LU) $A_{21} := (A_{21} - A_{20} A_{01}) U_{11}^{-1}$ (GEMM, TRSM) Variant 4: $A_{11} := \text{LU_UNB}(A_{11} - A_{10} A_{01})$ (GEMM, LU) $A_{21} := (A_{21} - A_{20} A_{01}) U_{11}^{-1}$ (GEMM, TRSM) $A_{12} := L_{11}^{-1} (A_{12} - A_{10} A_{02})$ (GEMM, LU) Variant 5: $A_{11} := \text{LU_UNB}(A_{11})$ (LU) $A_{21} := A_{21} U_{11}^{-1}$ (TRSM) $A_{12} := L_{11}^{-1} A_{12}$ (TRSM) $A_{22} := A_{22} - A_{21} A_{12}$ (GEMM) Continue with $\begin{pmatrix} A_{TL} & A_{TR} \\ A_{BL} & A_{BR} \end{pmatrix} \leftarrow \begin{pmatrix} A_{00} & A_{01} & A_{02} \\ A_{10} & A_{11} & A_{12} \\ A_{20} & A_{21} & A_{22} \end{pmatrix}$ endwhile
--	---

BLAS1 – vector_vector

BLAS3 – matrix multiplications

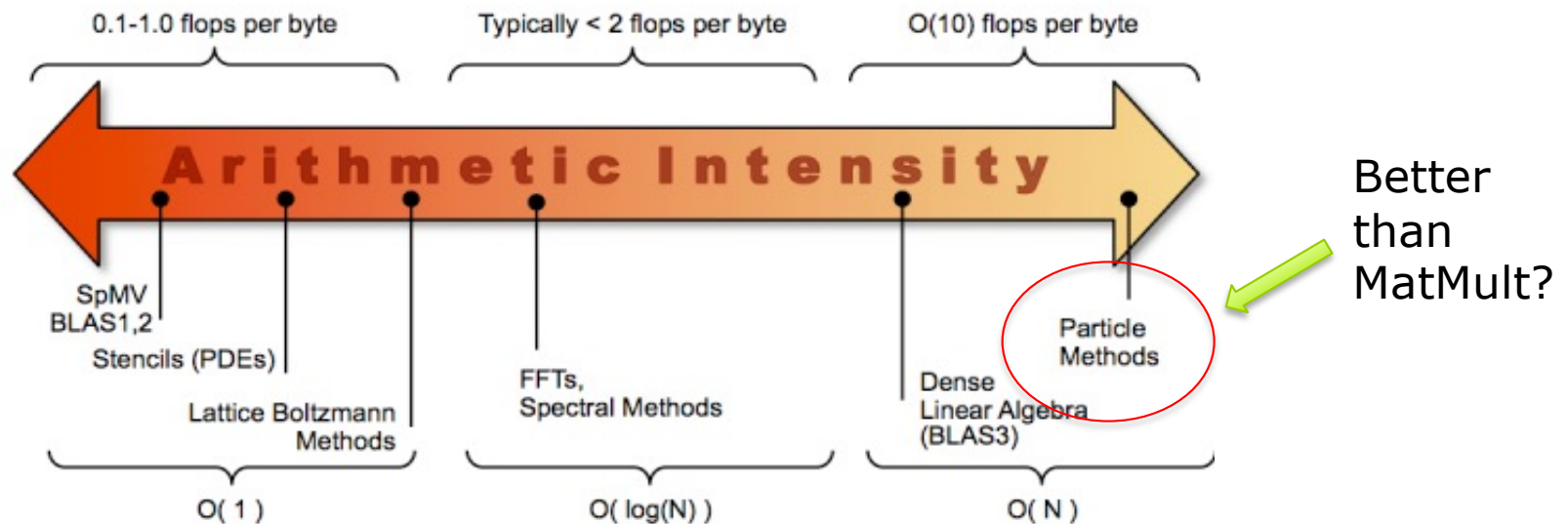
Hardware support

Intel AMX & NVIDIA Tensor Cores



Parallel Algorithms

□ Arithmetic Intensity – Molecular Dynamics



□ Data storage

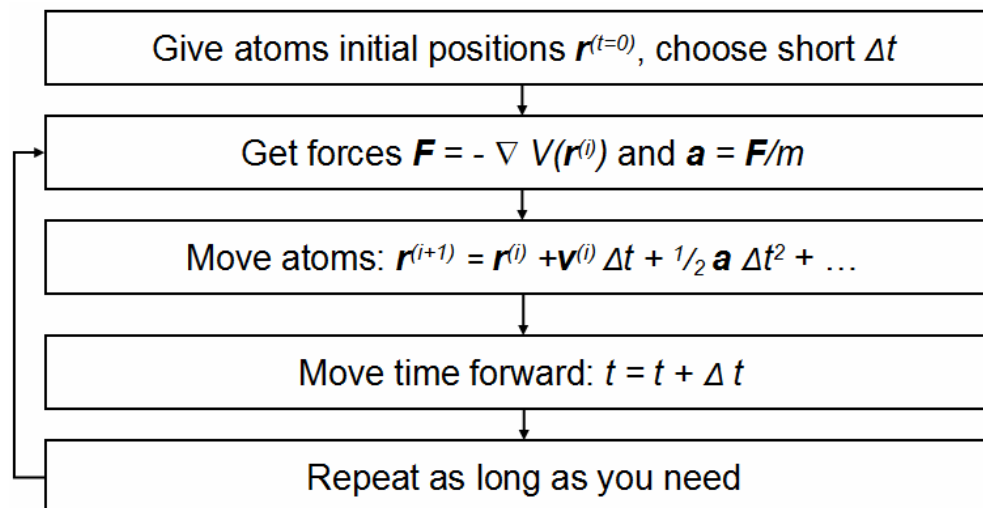
- **Regular** : `vector<Particles>`

- **Algorithm optimisations:**

- Irregular: `map<Particle, Set<Particle>>` → Sparse matrix

Molecular Dynamics (MD) Simulations

- **A molecular dynamics simulation aims to compute thermodynamic properties of a material**
 - Chemistry: study of proteins (folding and kinetics)
 - Physics: thin-films
 - Nanotechnologies: device simulations
- **The dynamics can be approximated by classical mechanics**
 - N particles are moving according to the laws of Newton's mechanics, with each particle feeling a force generated by the N-1 other particles
- **Simplified code skeleton of a simulation**



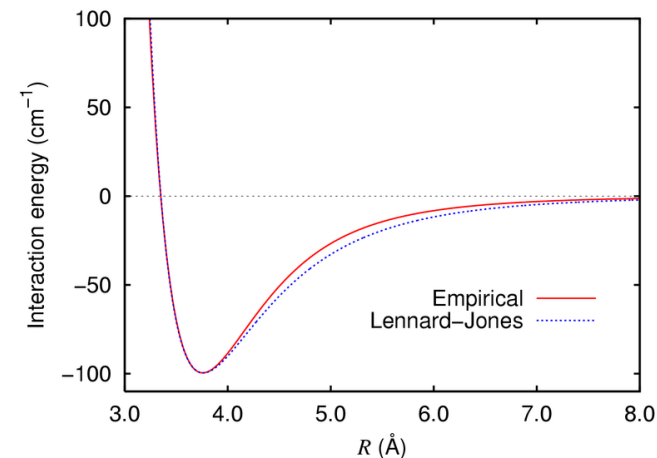
MD Simulations

Lennard-Jones potential

- Simplified model to describe interactions among particles:

$$\phi(r) = 4\epsilon \left[\left(\frac{\sigma}{r} \right)^{12} - \left(\frac{\sigma}{r} \right)^6 \right]$$

- r – distance between particles
- ϵ = field depth
- σ = distance where the field is null
- ϵ and σ are specific for each material (commonly set to 1 in simulations)



MD Simulations

Simulation Model (simplified)

- The system energy is the sum of Potential and Kinetic energy
 - The potential energy (V) is the sum the interactions between all pairs of particles

$$V(r_1, \dots, r_N) = \sum_i \sum_{j>i} \phi(|r_i - r_j|)$$

- Kinetic energy is computed using the particles velocity

$$K(t) = \frac{1}{2} \sum_i m_i [v_i(t)]^2$$

- Forces between particles are given by the gradient of the potential (e.g., Lennard-Jones potential)

$$\mathbf{F}(r) = -\nabla V(r) = -\frac{d}{dr} V(r) \hat{\mathbf{r}} = 4\epsilon \left(12 \frac{\sigma^{12}}{r^{13}} - 6 \frac{\sigma^6}{r^7} \right) \hat{\mathbf{r}}$$

- These equations cannot be solved exactly but need to be numerically solved. One commonly used method is the Verlet Integration algorithm:

$$\mathbf{r}(t + \Delta t) = \mathbf{r}(t) + \mathbf{v}(t)\Delta t + (1/2)\mathbf{a}(t)\Delta t^2$$

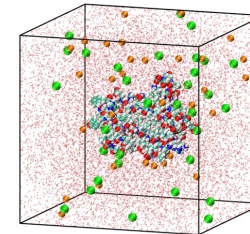
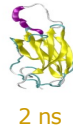
$$\mathbf{v}(t + \Delta t) = \mathbf{v}(t + \Delta t/2) + (1/2)\mathbf{a}(t + \Delta t)\Delta t$$

- How to deal with the simulation borders?
 - Replicate the simulation (*periodic boundary conditions*)
 - Other approaches

MD Simulations

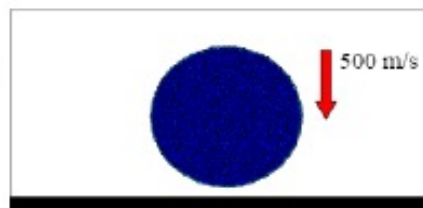
Simulation examples

- Satellite Tobacco Mosaic Virus – one of the smallest reproducing units in nature
 - 2006, 1 million atoms for over 50 ns
 - NAMD software (would require 35 years of simulation on a desktop computer at the time)
- P-Found – L55P-TTR - Transtirretina, “Doença dos Pezinhos” (1:1000 in Portugal)
 - **2004, 44k atoms** (Protein: 1912 atoms; Water: 3*14137 atoms; Na⁺ Cl⁻: 71 ions)
 - **Computation run time: 4-6 weeks** using 8-12 Pentium-4 CPU
 - **Binary file capturing the coordinates of all atoms: ≈ 4 GB**
 - **Binary file capturing the coordinates of protein's atoms: ≈**

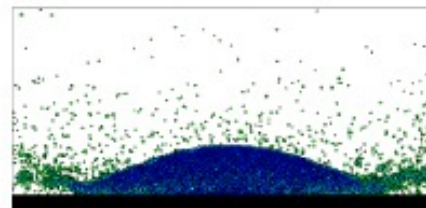


Molecular Dynamics

Example: collision of a droplet with a substrate
(by Yasushi Katsumi)



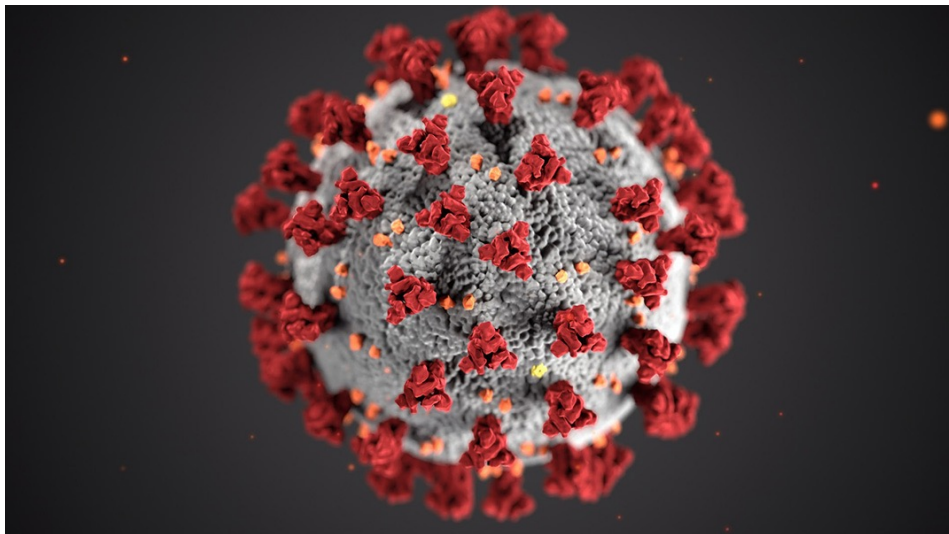
Initial conditions are specified,
 $r_i(t_0)$ and $v_i(t_0)$



Snapshot from MD simulation at
time $t_n = 100$ ps

MD Simulations

❑ Supercomputer simulates molecular model of SARS-CoV-2



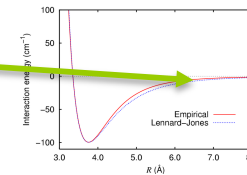
- [Rommie Amaro](#) is leading efforts to build the first complete **all-atom model** of the SARS-COV-2 coronavirus envelope
- The coronavirus model is anticipated contain roughly **200 million atoms**
- The Frontera supercomputer aided efforts of the Amaro Lab on March 12-13, 2020, by running [NAMD](#) molecular dynamics simulations on up to 4,000 nodes, or about **250,000** cores.

MD Simulations

□ Optimizations to the sequential code

■ Domain specific

- 3rd newton law – Forces are symmetric
- Cut-off radius – forces can be considered null at a long range
 - Neighbours list
 - Cell division



■ Generic

- Improve spatial and temporal locality
- Vectorization

■ Side

viewing AoS to SoA
as a matrix transpose

• AoS

- Most logical layout
- Difficult for gather reads and scatter writes, difficult for vectorization
- May lead to better cache utilization if data is accessed randomly

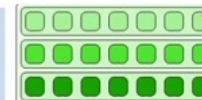
```
struct AOS_Point {  
    float x, y, z;  
};  
struct AOS_Point points[1024];
```



• SoA

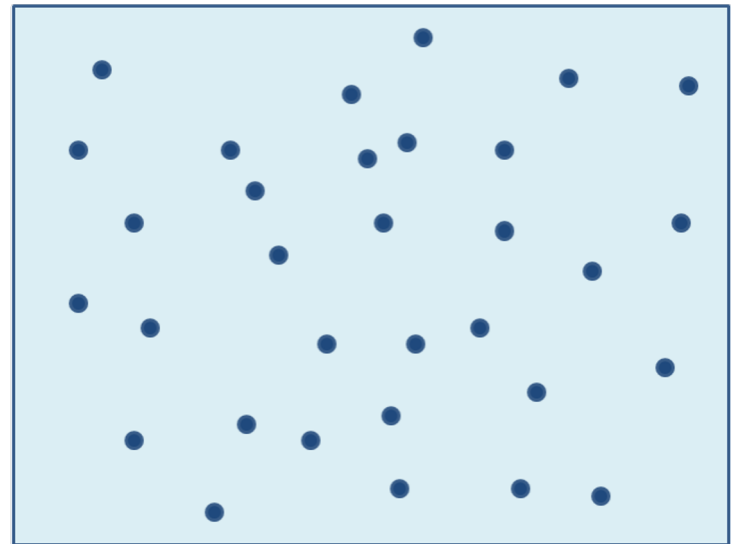
- Your data are organized in a way that if you have a struct with x,y,z, you have all the x all together and the same for the other variables
- Separate each structure field
- Keep memory access contiguous if access over structure instances

```
struct SoA_point {  
    float x[1024], y[1024], z[1024];  
};  
struct SoA_point Points;
```



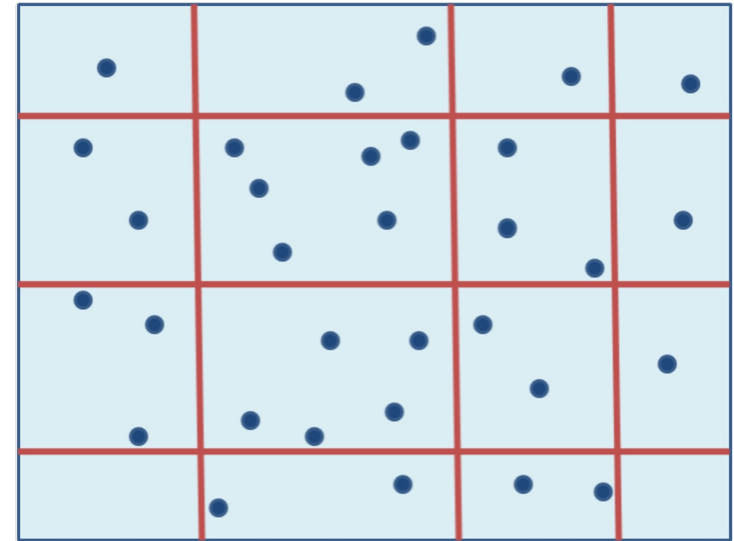
Base implementation - All pairs

- All particles interact with all others
- Ease of implementation
- Complexity N^2



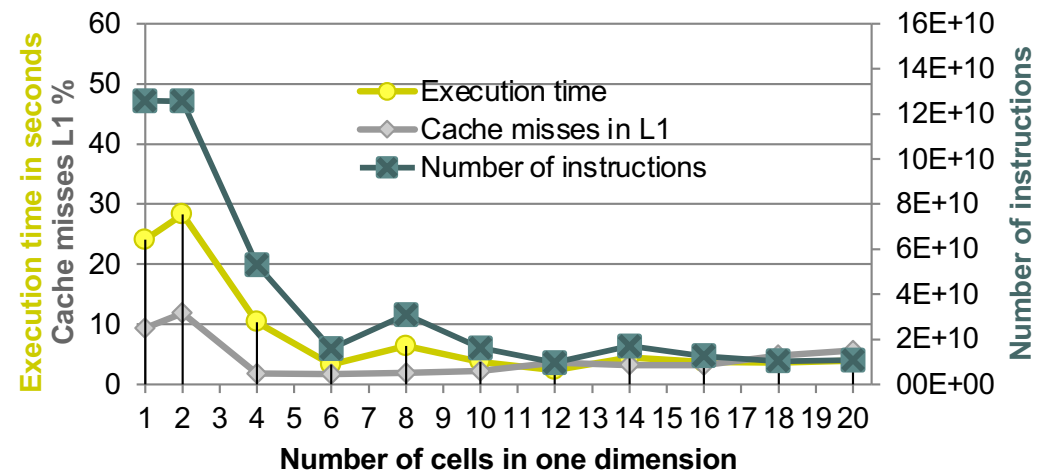
Cell Division (domain decomposition)

- Partition of particles in space
- Reduces the calculations when compared with all-2-all approach
- Improves locality of memory accesses
 - if “2 x dim(Cell)” fits in cache
- Must deal with Particle moves across cells

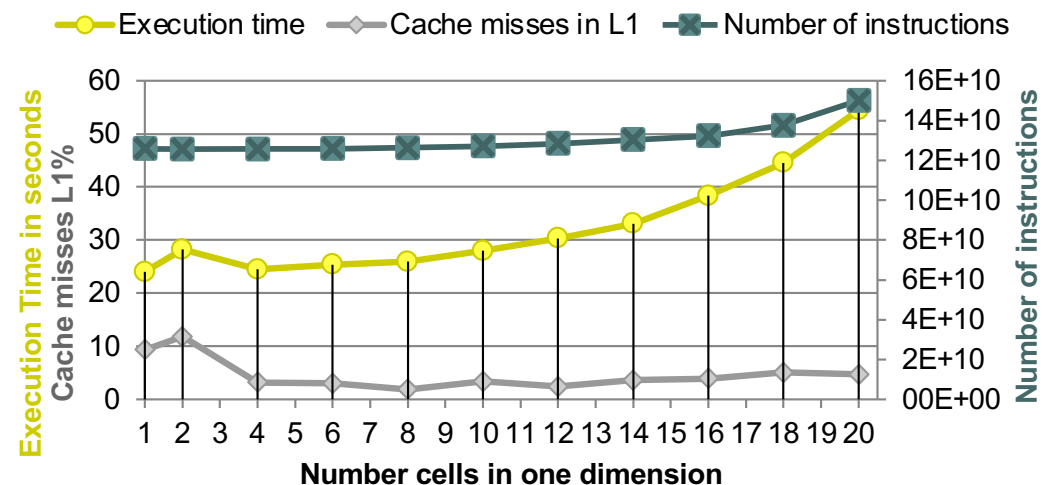


Cells optimization profiling

- Reduction of cache misses
- Decreases the number of instructions
- Minimum execution time when cell size is multiple of cut-off radius



Cells(All-with-All)



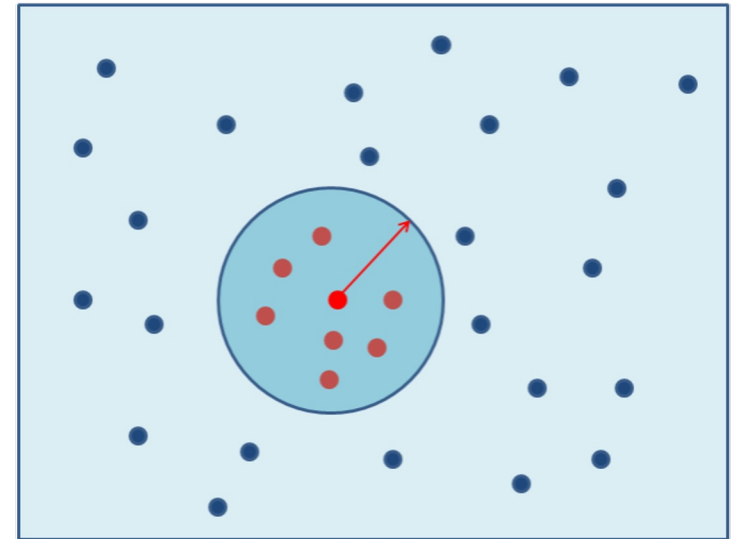
Neighbours list

Every particle contains one list of neighbour particles

Further reduces the calculations

It is not necessary to update the list at all iterations

- Use an extra radius delta
- Reduces overhead of list maintenance



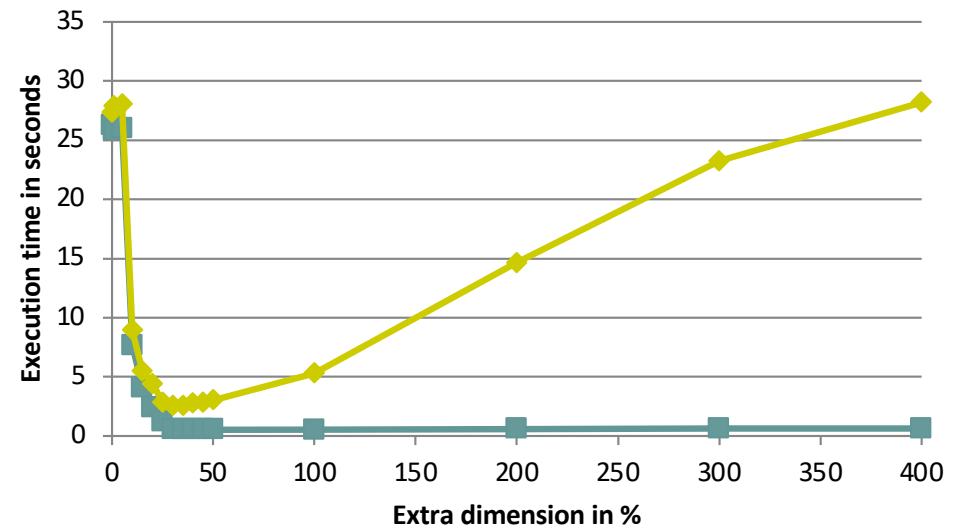
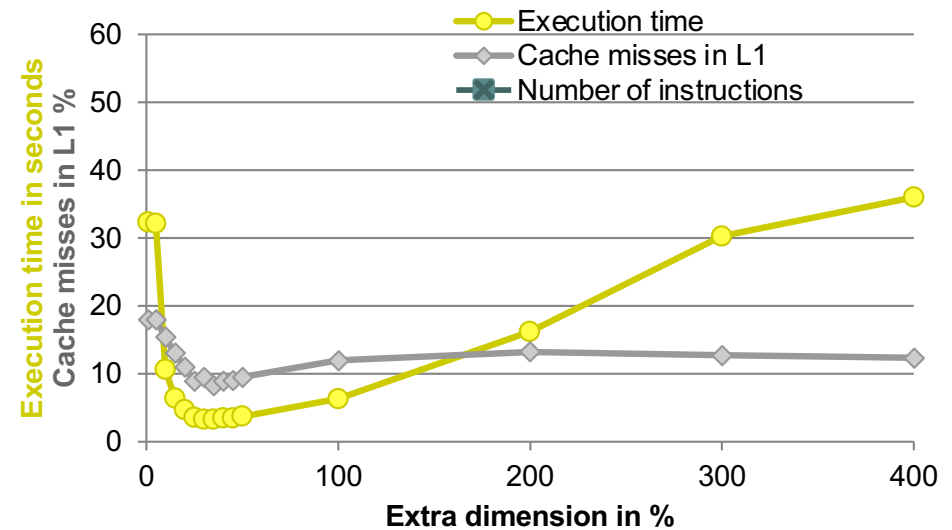
- Irregular data structure
Map<Particle,Set<Particle>>

Neighbour list optimization profiling

High number of
cache misses

Decreases
number of
instructions

List update can
introduce an
high overhead



Molecular Dynamics Simulation

Options to explore parallelism

- Decomposition of the set of particles
- Decomposition of the set of forces
- Domain decomposition (cells)

Parallel Implementations

- NAMD
- GROMACS
- MOIL

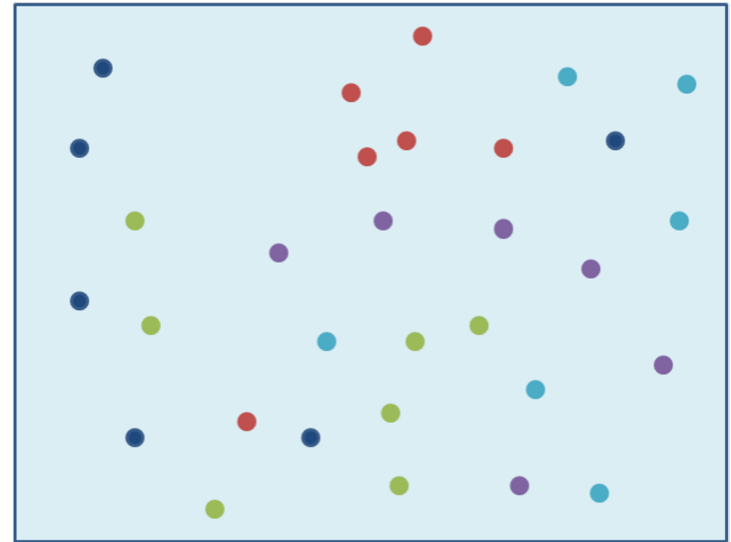
Particle decomposition

The particles are associated with a processing element

Different particles have different numbers of interactions

Each PE must know the positions of all particles

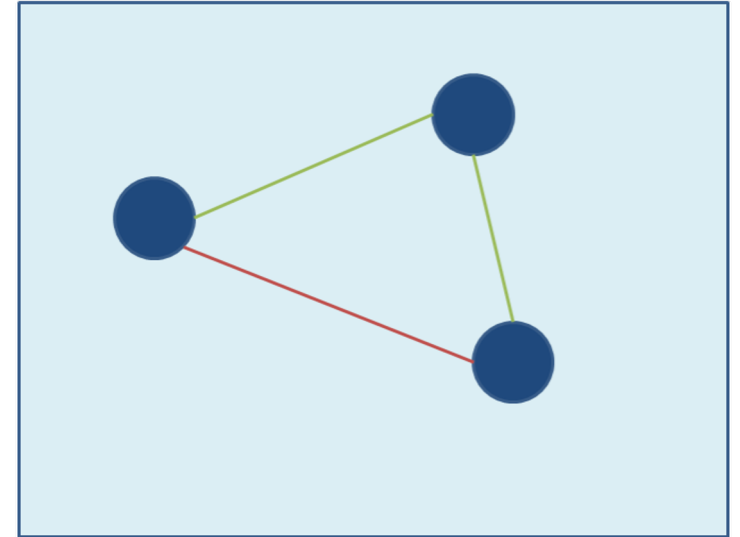
(requires global communication)



Force decomposition

Division of interactions
among PEs

Good load balancing



Space (cells) decomposition

The partition is based on the cell division

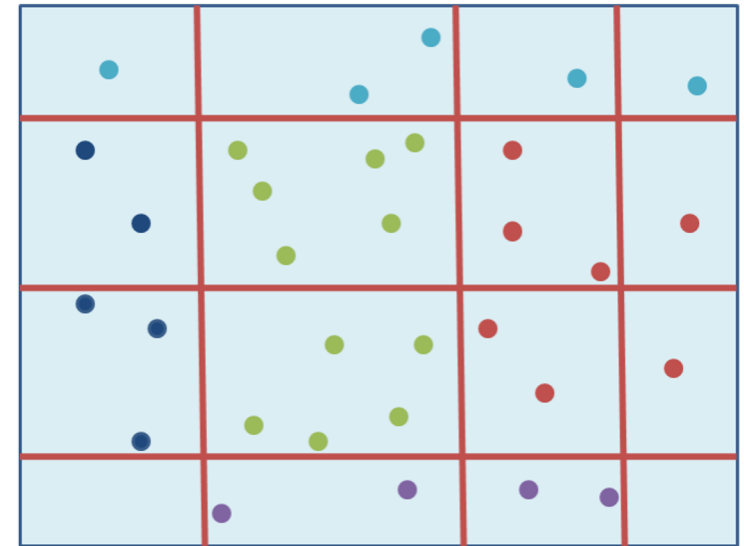
Each PE is assigned to one or more cells

Each cell needs the positions of the particles from neighbouring cells

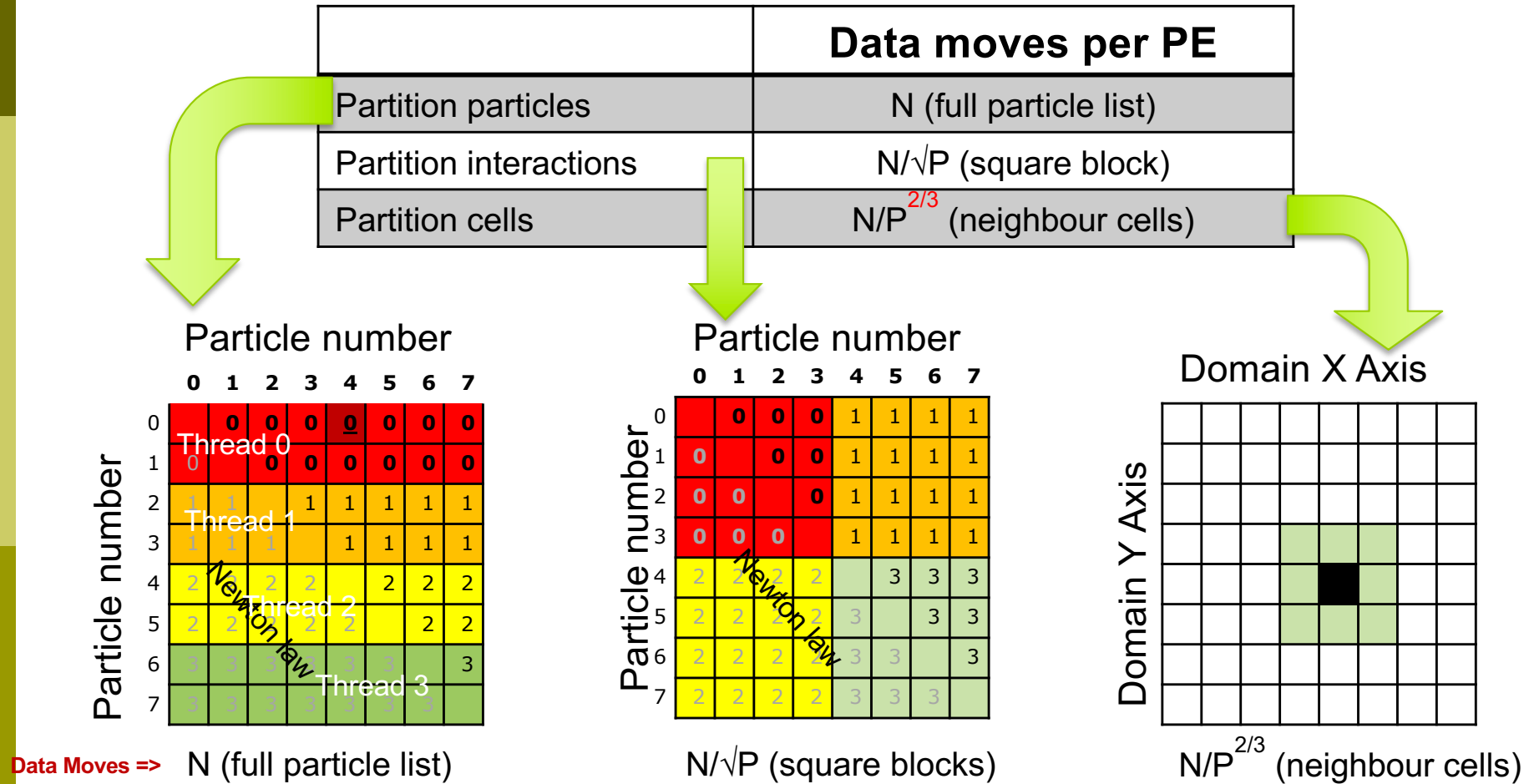
- depends on the cut-off/cell ratio

Particle migration among PE

Problems with load balancing



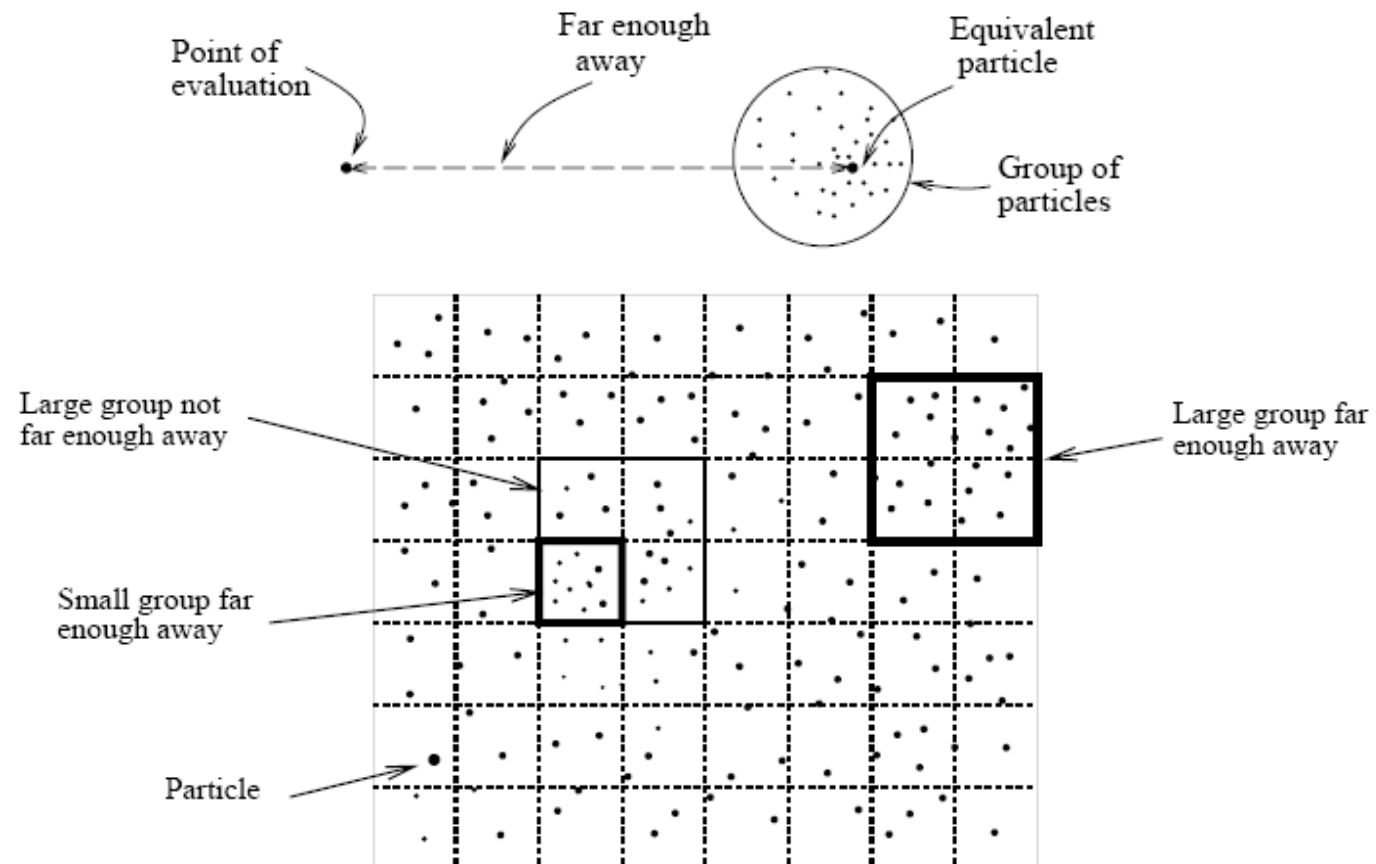
Parallel algorithm complexity



Force matrix: F_{ij} is the computation of force between particle i and particle j . In the left example F_{04} is computed by Thread 0.
 Newton Law states that forces are symmetric $F_{04}=F_{40}$

N-Body Methods

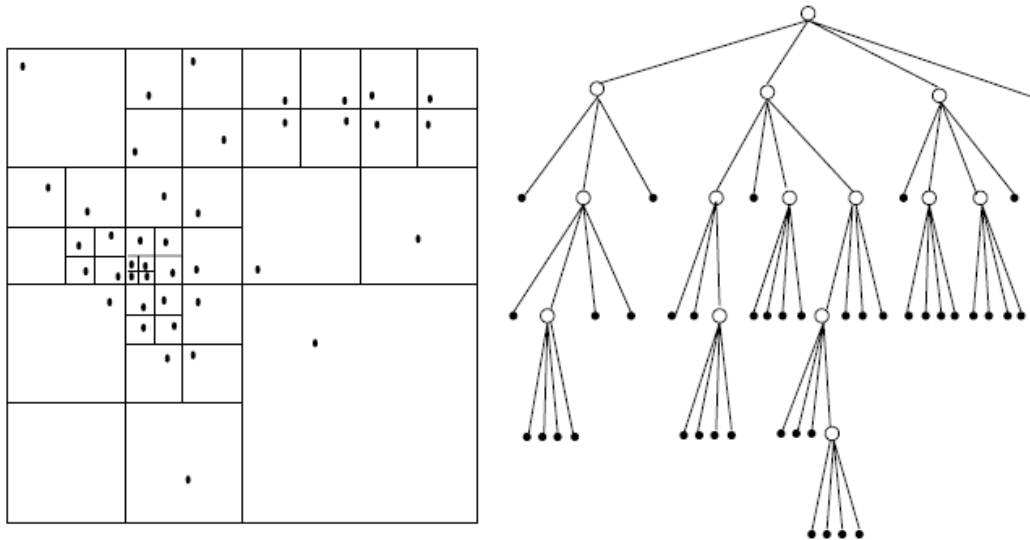
Distant points have less influence for force calculation



N-Body Methods

Barnes-Hut

- The space is subdivided until there is only one particle per cell.



- The force computation becomes a tree traversal
 - **Close enough particles are directly computed (leaf to leaf)**
 - **Far away particles are replaced by the root of their tree**
- Parallelism
 - **On force computation (the tree does not change during computation)**
 - **Tree reorganization**

Hierarchical N-Body Methods

□ **Parallelisation inefficiencies**

- Sequential computations
- Redundant work
- Managing parallelism
- Synchronization
- Communication
- Poor load distribution

□ **Trade-off between load distribution and data access locality**

- The unit of parallelism per cell (or particle) is irregular (depends on the distribution of particles)
 - Partitions with the same number of particles do not have the same workload
 - Solution: Assign a cost function to the calculations associated with each particle and use that cost to distribute the load
- Communication becomes fine-grained, global, and unstructured
- The distribution of the particles varies during execution (slowly)
 - Static partition does not guarantee physical locality (nearby particles in space must be assigned to the same processor)
 - Solution: Partition the tree

PL lecture

□ **Parallelize the given MD simulation code**

- Shared Memory: Use OpenMP & GCC
- [partial] correctness is verified

□ **Suggestion**

- Profile the code execution and identify the function responsible by the force computation
- Identify the three lines responsible by the 3rd newton law optimization inside the force computation
- Start with a naive OpenMP particle-based parallelisation taking care of the data races
- Optimize the parallel implementation (optional)

Particle methods

Skeleton of the provided code

DataTypes.h

```
typedef struct MD{
    int ITERS,size,interactions, mdsiz, move, mm;
    int irep,istop ,iprint ,movemx;

    double LENGTH,m,mu,kb,TSIM,deltat,epot,vir;
    double l, rcoeff, rcoeffs, side, sideh, hsq, hsq2;
    double a, r, tscale, sc,ek, ts;
    double den,tref,h;

    double vaver, vaverh, rand2;
    double etot, temp, pres, rp;
    double sum,ekin,count,vel;
}MD;

typedef struct PARTICLE{
    double *posX,*posY,*posZ; // Posiões
    double *vX, *vY, *vZ; // Velocidades
    double *fX, *fY, *fZ; // Forças
} Particles;
```

MD.C

```
void runiters(MD *md, Particles *part){

    for (md->move = 0; md->move < md->movemx; md->move++) {

        cicleDoMove      (md,part); // Calcular o movimento
        cicleForces      (md,part); // Calcular a força
        cicleMkekin      (md,part); // Scale forces, update velocities
        cicleVelavg      (md,part); // calcular a velocidade
        scale_temperature (md,part); // temperature scale if required
        get_full_potential_energy(md); // sum to get full potential energy and virial
    }
}

/* move the particles and update velocities */
void cicleDoMove(MD *md, Particles *part){
    for (int i = 0; i < md->mdsiz; i++) domove(md->side,part,i);
}

/* compute forces */
void cicleForces(MD *md,Particles *part){

    md->epot = md->vir = 0.0;

    for (int i = 0; i < md->mdsiz; i++)
        force(md,part,i);
}
```